

MediCloud

Guide Complet de Déploiement

Du clonage GitHub au déploiement Vercel

Ce guide a été rédigé pour permettre à n'importe quel développeur de **lancer MediCloud localement** et de le **déployer en production** sur Vercel, pas à pas, depuis zéro.

Auteur : Noaman Makhoul

Dépôt GitHub : github.com/Noamane2023/medicloud

Stack technique : Next.js 16 · Supabase · Upstash Redis · Vercel

Date : 18 avril 2026

Table des matières

1	Introduction — Qu'est-ce que MediCloud ?	2
2	Étape 1 — Prérequis : Logiciels à Installer	2
2.1	Node.js (v18 ou supérieur)	2
2.2	Git	2
2.3	Visual Studio Code (éditeur recommandé)	3
3	Étape 2 — Créer un Compte GitHub et Accéder au Dépôt	3
3.1	Créer un compte GitHub	3
3.2	Accepter l'invitation de collaboration	3
4	Étape 3 — Cloner et Lancer le Projet Localement	3
4.1	Cloner le dépôt	3
4.2	Installer les dépendances	4
4.3	Configurer les variables d'environnement	4
4.4	Lancer le serveur de développement	4
5	Étape 4 — Déploiement en Production sur Vercel	5
5.1	Créer un compte Vercel	5
5.2	Importer le projet depuis GitHub	5
5.3	Ajouter les variables d'environnement	5
5.4	Lancer le déploiement	5
5.5	Déploiement Continu (CD)	6
6	Étape 5 — Vérification Post-Déploiement	6
6.1	Checklist de vérification	6
6.2	Tester les Cron Jobs manuellement	6
7	Ce que tu Devrais Apprendre — Feuille de Route	6
7.1	Fondations (indispensables)	6
7.2	Back-end & Bases de Données	7
7.3	Cloud & DevOps	7
7.4	Concepts Cloud Native avancés (pour aller plus loin)	7
8	Ressources et Références	7

1. Introduction — Qu'est-ce que MediCloud ?

MediCloud est une plateforme de télémédecine Cloud-Native développée dans le cadre d'un projet académique à l'ENSEM. Elle permet à des patients de trouver un médecin, de réserver un rendez-vous en ligne, et de gérer un dossier médical numérique — le tout depuis un smartphone ou un navigateur.

Architecture technique en bref

Couche	Technologie	Rôle
Front-end	Next.js 16 + React	Interface utilisateur PWA
Back-end	API Routes (Serverless)	Logique métier
Base de données	Supabase (PostgreSQL)	Données persistantes
Cache	Upstash Redis	Réponses rapides sous charge
Emails	Resend API	Rappels de rendez-vous
Déploiement	Vercel	Hébergement Cloud Edge

2. Étape 1 — Prérequis : Logiciels à Installer

Avant de commencer, tu dois avoir installé les outils suivants sur ton ordinateur (Windows, macOS ou Linux).

2.1. Node.js (v18 ou supérieur)

Node.js est le moteur d'exécution JavaScript utilisé par Next.js.

1. Rends-toi sur nodejs.org
2. Télécharge la version **LTS** (Long Term Support)
3. Installe-le en gardant les options par défaut
4. Vérifie l'installation en ouvrant un terminal et en tapant :

```
node --version
# Attendu : v18.x.x ou supérieur

npm --version
# Attendu : 9.x.x ou supérieur
```

2.2. Git

Git est le système de contrôle de version utilisé pour télécharger le code depuis GitHub.

1. Rends-toi sur git-scm.com/downloads
2. Télécharge et installe Git selon ton système d'exploitation
3. Vérifie l'installation :

```
git --version
# Attendu : git version 2.x.x
```

2.3. Visual Studio Code (éditeur recommandé)

code.visualstudio.com — Télécharge et installe VS Code. C'est l'éditeur de code le plus populaire pour le développement web moderne.

Extensions recommandées à installer dans VS Code :

- **ESLint** — Détection d'erreurs JavaScript/TypeScript
- **Tailwind CSS IntelliSense** — Autocomplétion CSS
- **Prettier** — Formatage automatique du code
- **GitLens** — Visualisation Git avancée

3. Étape 2 — Créer un Compte GitHub et Accéder au Dépôt

3.1. Créer un compte GitHub

1. Va sur github.com
2. Clique sur "**Sign up**"
3. Remplis le formulaire : adresse email, mot de passe, nom d'utilisateur
4. Confirme ton email
5. Envoie ton **nom d'utilisateur GitHub** à Noaman pour être ajouté comme collaborateur sur le dépôt

Important — Communique ton pseudonyme GitHub

Dis à Noaman ton nom d'utilisateur GitHub (pas ton email) pour qu'il t'ajoute aux collaborateurs du dépôt. Tu recevras une invitation par email à accepter.

3.2. Accepter l'invitation de collaboration

1. Vérifie ta boîte email
2. Clique sur le lien d'**invitation GitHub**
3. Accepte l'accès au dépôt Noamane2023/medicloud

4. Étape 3 — Cloner et Lancer le Projet Localement

4.1. Cloner le dépôt

Ouvre un terminal (PowerShell, Terminal macOS ou Bash Linux) et exécute :

```
# Navigue vers le dossier de ton choix
cd Documents
```

```
# Clone le depot depuis GitHub
git clone https://github.com/Noamane2023/medicloud.git

# Entre dans le dossier du projet
cd medicloud
```

4.2. Installer les dépendances

```
# Installe tous les packages nécessaires
npm install
```

Cette commande va télécharger toutes les bibliothèques listées dans le fichier `package.json` (React, Next.js, Supabase SDK, etc.).

4.3. Configurer les variables d'environnement

Le projet a besoin de clés secrètes pour se connecter aux services en ligne. Ces clés ne sont **jamais** publiées sur GitHub pour des raisons de sécurité.

1. Crée un fichier `.env.local` à la racine du projet
2. Demande à Noaman de te partager les valeurs privées
3. Remplis le fichier ainsi :

```
# .env.local - Ne jamais partager ce fichier !

NEXT_PUBLIC_SUPABASE_URL=https://xxxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOiJ...
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOiJ...

RESEND_API_KEY=re_xxxxxxxxxxxxxx
CRON_SECRET=medicloud_cron_2026

UPSTASH_REDIS_REST_URL=https://xxx.upstash.io
UPSTASH_REDIS_REST_TOKEN=xxxxxxxxxx
```

4.4. Lancer le serveur de développement

```
npm run dev
```

Succès !

Ouvre ton navigateur sur <http://localhost:3000>. Tu devrais voir la page d'accueil de MediCloud.

Le serveur local se rechargera automatiquement à chaque modification du code.

5. Étape 4 — Déploiement en Production sur Vercel

Vercel est la plateforme Cloud officielle de déploiement pour les applications Next.js. Elle offre un **déploiement continu** : chaque push sur GitHub met à jour automatiquement l'application en production.

5.1. Créer un compte Vercel

1. Va sur vercel.com
2. Clique sur **"Sign Up"**
3. Authentifie-toi avec ton **compte GitHub** (recommandé)

5.2. Importer le projet depuis GitHub

1. Dans le Dashboard Vercel, clique sur **"Add New... → Project"**
2. Sélectionne le dépôt `medicloud` depuis la liste GitHub
3. Vercel détecte automatiquement qu'il s'agit d'un projet **Next.js**
4. **Ne clique pas encore sur "Deploy"** — il faut d'abord ajouter les variables

5.3. Ajouter les variables d'environnement

Dans la section **"Environment Variables"** de la page de configuration Vercel :

Nom de la variable	Description
<code>NEXT_PUBLIC_SUPABASE_URL</code>	URL de ta base de données Supabase
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Clé publique Supabase
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Clé admin secrète Supabase
<code>RESEND_API_KEY</code>	Clé API pour l'envoi d'emails
<code>CRON_SECRET</code>	Mot de passe pour sécuriser les Cron Jobs
<code>UPSTASH_REDIS_REST_URL</code>	URL de la base de données Redis
<code>UPSTASH_REDIS_REST_TOKEN</code>	Token d'authentification Redis

Attention à la sécurité

Les variables préfixées `NEXT_PUBLIC_` sont visibles par le navigateur. Ne mets **jamais** de données sensibles (clés secrètes) dans des variables `NEXT_PUBLIC_`. Supabase et Vercel gèrent cela automatiquement.

5.4. Lancer le déploiement

1. Clique sur **"Deploy"**
2. Vercel va compiler et déployer l'application (environ 2 à 4 minutes)
3. À la fin, tu obtiens une URL publique du type : `https://medicloud-xxxx.vercel.app`

5.5. Déploiement Continu (CD)

Après la configuration initiale, chaque fois que quelqu'un push du code sur la branche main de GitHub, Vercel **redéploie automatiquement** l'application sans aucune action manuelle.

```
# Workflow de mise a jour standard
git add .
git commit -m "feat: nouvelle fonctionnalite"
git push origin main
# -> Vercel detecte le push et relance le deployment
      automatiquement
```

6. Étape 5 — Vérification Post-Déploiement

6.1. Checklist de vérification

- ✓ L'URL Vercel s'ouvre et affiche la page d'accueil
- ✓ La page /login fonctionne et permet la connexion
- ✓ La recherche de médecins (/patient/search) affiche des résultats
- ✓ Dans l'onglet **Cron** du Dashboard Vercel, les tâches planifiées sont visibles et actives
- ✓ Dans Chrome DevTools → Application → Service Workers, le worker est actif ("Activated and running")

6.2. Tester les Cron Jobs manuellement

Pour vérifier que les tâches automatiques fonctionnent, appelle ces URLs (avec un token d'autorisation) :

```
# Rappels email (simulation)
https://ton-app.vercel.app/api/cron/reminders?bypass=true

# Nettoyage automatique de la BDD
https://ton-app.vercel.app/api/cron/cleanup?bypass=true
```

7. Ce que tu Devrais Apprendre — Feuille de Route

Pour comprendre en profondeur comment ce projet fonctionne et progresser en développement web Cloud, voici les compétences clés à acquérir :

7.1. Fondations (indispensables)

- **JavaScript/TypeScript** — Le langage de base de tout le projet. Ressource : javascript.info
- **React.js** — La bibliothèque qui construit l'interface. Ressource : react.dev
- **Next.js** — Le framework Full-Stack qui orchestre tout. Ressource : nextjs.org/learn

- **Git & GitHub** — Contrôle de version et collaboration. Ressource : learngitbranching.js.org

7.2. Back-end & Bases de Données

- **SQL & PostgreSQL** — Comprendre les requêtes, les tables, les jointures.
- **Supabase** — BaaS (Backend as a Service) : Auth, BDD, Storage. Ressource : supabase.com/docs
- **API REST** — Comment concevoir et consommer des APIs HTTP.
- **Authentification JWT** — Tokens, sessions, sécurité.

7.3. Cloud & DevOps

- **Vercel** — Déploiement, CI/CD, Serverless Functions, Cron Jobs. Ressource : vercel.com/docs
- **Variables d'environnement** — Gestion des secrets en production.
- **DNS & Domaines** — Comment configurer un domaine personnalisé.
- **Redis (Upstash)** — Cache distribué pour la scalabilité.

7.4. Concepts Cloud Native avancés (pour aller plus loin)

- **Progressive Web Apps (PWA)** — Service Workers, Manifestes, Mode hors-ligne.
- **Serverless Architecture** — Functions as a Service (FaaS), Cron Jobs.
- **Edge Computing** — Exécuter du code au plus près de l'utilisateur.
- **Row Level Security (RLS)** — Sécurité au niveau des lignes de BDD.
- **CI/CD Pipelines** — Déploiement automatisé à chaque commit.

8. Ressources et Références

Ressource	Lien	Usage
Code source	github.com/Noamane2023/medicloud	Accès au projet
Next.js Docs	nextjs.org/docs	Framework
Supabase Docs	supabase.com/docs	Base de données
Vercel Docs	vercel.com/docs	Déploiement
Upstash Docs	docs.upstash.com	Redis Cache
Resend Docs	resend.com/docs	Emails

Conclusion

En suivant ce guide, tu as pu :

- Installer l'environnement de développement complet
- Lancer MediCloud localement sur ton ordinateur
- Déployer l'application en production sur Vercel
- Comprendre la feuille de route pour maîtriser ces technologies

Bravo !

Le développement Cloud Native est l'une des compétences les plus demandées dans l'industrie tech. Les technologies utilisées dans ce projet (Next.js, Supabase, Vercel, Redis) sont utilisées par des milliers d'entreprises à travers le monde. Continue à explorer !

Document rédigé avec $\LaTeX$ par Noaman Makhlouf — ENSEM 2026